

Table of Contents

I. INTRODUCTION.....	1
II. A PRACTICAL C PROGRAM: PROCESS INFORMATION.....	2
III. THE C STANDARD LIBRARY: TWO HALVES.....	3
A. Part 1: Header Files — The Construction Manual.....	3
B. Part 2: Precompiled Machine Code — The Actual Libraries.....	4
C. Summary: The Two-Part Anatomy.....	5
IV. STATIC VS. DYNAMIC LINKING.....	5
A. Static Linking.....	5
B. Dynamic Linking.....	6
C. Why It Matters?.....	6
V. INSPECTING EXECUTABLES WITH READ ELF.....	6
A. Three Views of an Executable.....	6
B. View 1: The ELF Header (readelf -h).....	7
C. View 2: The Program Headers (readelf -l).....	8
D. View 3: The Dynamic Section (readelf -d).....	9
VI. TRACING DEPENDENCIES WITH L D D.....	10
A. Static Binaries and ldd.....	10
VII. COMMAND REFERENCE SUMMARY.....	11
VIII. CONCLUSION.....	12
IX. KNOWLEDGE CHECK QUESTIONS.....	13

List of Figures

Figure 1: procinfo.c.....	3
Figure 2: Include Directory.....	5
Figure 3: Two Part Anatomy.....	6
Figure 2: Two parts of the C Standard Library.....	6
Figure 4: procinfo_static.....	6
Figure 3: procinfo_dynamic.....	7
Figure 4: Viewing the ELF Header of procinfo_dynamic.....	8
Figure 5: Viewing the Program Header of procinfo_dynamic.....	9
Figure 6: readelf -d.....	10
Figure 7: ldd function.....	11
Figure 8: ldd function on static.....	11
Figure 9: TABLE II: Command Reference: Inspecting ELF Executables.....	12

Lab 2: C Libraries, Linking, and ELF Executable Structure

I. INTRODUCTION

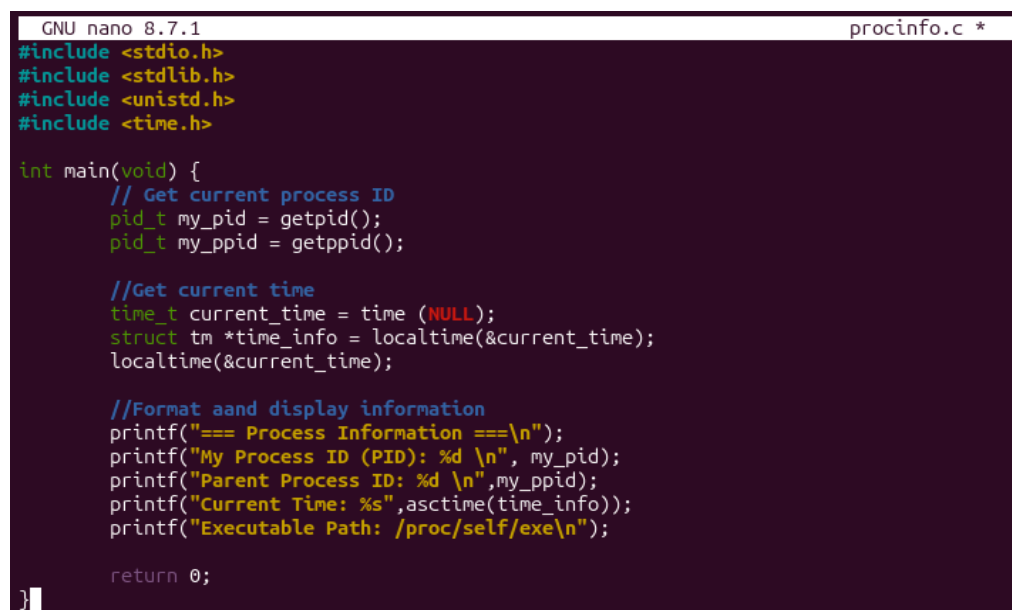
When a programmer writes a C program that uses library functions like `printf()`, `getpid()`, and `time()`, compiles it, and runs it, a remarkable chain of events occurs. The compiler transforms human-readable C code into machine instructions. The linker resolves references to external libraries like `libc`. The operating system reads the resulting binary and maps it into virtual memory.

This lab answers three concrete questions:

- 1) How does a C program link to the Standard Library?
- 2) Where do C library headers and compiled code physically live on disk?
- 3) What is the difference between static and dynamic linking, and how can we inspect the resulting executables?

II. A PRACTICAL C PROGRAM: PROCESS INFORMATION

We begin with a C program that demonstrates real library usage by displaying process information. This program uses multiple functions from the C Standard Library and POSIX extensions:



```
GNU nano 8.7.1 procinfo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void) {
    // Get current process ID
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    //Get current time
    time_t current_time = time(NULL);
    struct tm *time_info = localtime(&current_time);
    localtime(&current_time);

    //Format aand display information
    printf("=== Process Information ===\n");
    printf("My Process ID (PID): %d \n", my_pid);
    printf("Parent Process ID: %d \n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable Path: /proc/self/exe\n");

    return 0;
}
```

Figure 1: `procinfo.c`

```

bipin@bipin-VirtualBox:~$ nano procinfo.c
bipin@bipin-VirtualBox:~$ gcc procinfo.c -o procinfo
bipin@bipin-VirtualBox:~$ ./procinfo
=== Process Information ===
My Process ID (PID): 6799
Parent Process ID: 3738
Current Time: Tue Oct  6 16:47:53 2026
Executable Path: /proc/self/exe
bipin@bipin-VirtualBox:~$

```

This program uses several library functions:

- getpid() and getppid() from <unistd.h>
- time() and localtime() from <time.h>
- printf() and asctime() from <stdio.h>

All of these functions are implemented in the C Standard Library (libc). But where does the actual machine code for these functions come from?

III. THE C STANDARD LIBRARY: TWO HALVES

Every C program implicitly depends on the C Standard Library, commonly called libc. On Linux, the GNU implementation is glibc. This library is split into two distinct physical components that serve fundamentally different purposes.

A. Part 1: Header Files — The Construction Manual

Header files (.h) contain declarations: function prototypes, macro definitions, type definitions (typedef), and structure definitions. They tell the compiler what functions exist and how to call them, but they contain no executable machine code.

Analogy: A header file is like a construction manual. It tells the architect (the compiler) the dimensions and interface of every component, but it does not contain the actual bricks and steel.

Physical location:

/usr/include/

```

bipin@bipin-VirtualBox:~$ cd /usr/include/

```

We can verify this on any Linux system:

```
bipin@bipin-VirtualBox:/usr/include$ ls /usr/include/stdio.h /usr/include/unistd.h
/usr/include/stdio.h /usr/include/unistd.h
bipin@bipin-VirtualBox:/usr/include$ ls
X11          crypt.h      features.h   grp.h        math.h       netipx       pthread.h    semaphore.
aio.h        ctype.h      fenv.h       gshadow.h    mcheck.h     netiucv      pty.h        setjmp.h
aliases.h    cxl          finclude     iconv.h      memory.h     netpacket    pwd.h        sgTTY.h
alloca.h     dirent.h     fmtmsg.h     ifaddrs.h    nisc          netron       python3.14   shadow.h
ar.h         dlfcn.h     fnmatch.h    inttypes.h   mntent.h     netrose      rdma         signal.h
argp.h       drm          fstab.h      iproute2     monetary.h   nfs          re_comp.h    sound
argz.h       elf.h        fts.h        langinfo.h   mqueue.h     nl_types.h   regex.h      spawn.h
arpa         endian.h     ftw.h        lastlog.h    ntd           nss.h        regexp.h     stab.h
asm-generic  envz.h       fwctl        libgen.h     net           obstack.h    regulator     stdbit.h
assert.h     err.h        gconv.h      libintl.h    netash        openvpn      resolv.h     stdc-prede
byteswap.h   errno.h      gdb          limits.h     netatalk      paths.h      rpc          stdint.h
c++          error.h      getopt.h     link.h        netax25       poll.h       rpcsvc       stdio.h
complex.h    execinfo.h   glob.h       locale.h     netdb.h       printf.h     sched.h      stdio_ext.
cpio.h       fcntl.h      gnu-versions.h locale.h     neteconet     proc_service.h scsi         stdlib.h
crash        features-time64.h gnumake.h    malloc.h     netinet       protocols    search.h     string.h
bipin@bipin-VirtualBox:/usr/include$ grep "printf" /usr/include/stdio.h | head -2
/* Maximum length of printf output for a NaN. */
extern int fprintf (FILE *__restrict __stream,
bipin@bipin-VirtualBox:/usr/include$ grep "getpid" /usr/include/unistd.h | head -2
extern __pid_t getpid (void) __THROW;
bipin@bipin-VirtualBox:/usr/include$
```

Figure 2: Include Directory

When you write `#include <stdio.h>`, the preprocessor literally copies the contents of this file into your source code before compilation begins. The compiler now knows that `printf()` exists and what arguments it expects, but it does not yet know where the actual code lives.

B. Part 2: Precompiled Machine Code — The Actual Libraries

The actual compiled machine instructions for functions like `printf()`, `getpid()`, `time()`, and `malloc()` live in

library files. These come in two forms:

- **Static libraries (.a):** Archives of object files. The linker copies the needed code directly into your executable.
- **Shared/Dynamic libraries (.so):** Standalone binaries that are loaded into memory at runtime by the dynamic linker. Multiple programs share a single copy in RAM.

Physical locations:

`/lib/x86_64-linux-gnu/`

`/usr/lib/x86_64-linux-gnu/`

`/usr/lib/`

`/lib/`

We can verify the presence of libc:

```
bipin@bipin-VirtualBox:~$ ls /lib/x86_64-linux-gnu/libc.so*
/lib/x86_64-linux-gnu/libc.so  /lib/x86_64-linux-gnu/libc.so.6
bipin@bipin-VirtualBox:~$ ls /lib/x86_64-linux-gnu/libc.a
/lib/x86_64-linux-gnu/libc.a
bipin@bipin-VirtualBox:~$
```

Figure 3: Two Part Anatomy

C. Summary: The Two-Part Anatomy

Component	Contains	Location
Headers (.h)	Declarations, prototypes, macros	/usr/include
Static lib (.a)	Precompiled object code (archive)	/usr/lib/
Shared lib (.so)	Precompiled position-independent code	/lib/

Figure 2: Two parts of the C Standard Library

IV. STATIC VS. DYNAMIC LINKING

Linking is the process of resolving symbolic references (e.g., the name “printf”) to actual memory addresses. The timing and method of this resolution define two fundamentally different approaches.

A. Static Linking

In static linking, the linker (ld) copies the required machine code from .a archive files directly into the final executable at compile time.

```
bipin@bipin-VirtualBox:~$ gcc -static procinfo.c -o procinfo_static
bipin@bipin-VirtualBox:~$ ls -lh procinfo_static
-rwxrwxr-x 1 bipin bipin 961K Oct  6 17:29 procinfo_static
```

Figure 4: procinfo_static

Characteristics:

- The executable is self-contained; it needs no external .so files to run.
- The binary is significantly larger (850KB vs 16KB) because it embeds all library code.
- Updating a library requires recompiling every program that uses it.
- Useful for embedded systems, containers, and environments where portability is critical.

B. Dynamic Linking

In dynamic linking, the executable contains only references to shared libraries. The actual code is loaded into memory at runtime by the dynamic linker (typically `/lib64/ld-linux-x86-64.so.2`).

```
bipin@bipin-VirtualBox:~$ gcc -static procinfo.c -o procinfo_dyanimc
bipin@bipin-VirtualBox:~$ ls -lh procinfo_dyanimc
-rwxrwxr-x 1 bipin bipin 961K Oct  6 17:30 procinfo_dyanimc
```

Figure 3: `procinfo_dynamic`

Characteristics:

- The executable is small; library code is shared in RAM across all running programs.
- Updating `libc.so` instantly benefits every program that uses it.
- The program will fail to start if a required `.so` file is missing (the infamous “shared library not found” error).
- This is the default on virtually all modern Linux systems.

C. Why It Matters?

Dynamic linking reduces memory consumption and disk usage across the entire system. However, it introduces runtime dependencies and potential version conflicts (“dependency hell”). Static linking eliminates these problems at the cost of larger binaries and no automatic security patching.

V. INSPECTING EXECUTABLES WITH READELF

On Linux, compiled executables use the ELF (Executable and Linkable Format) binary format. The `readelf` tool allows us to read the internal metadata of an ELF file without executing it.

A. Three Views of an Executable

An ELF binary can be understood through three complementary views:

- 1) ELF Header (`readelf -h`):** Global metadata about the file—is it a valid executable, and for which CPU?
- 2) Program Headers (`readelf -l`):** Instructions for the OS loader—which segments to map into memory and with what permissions.
- 3) Dynamic Section (`readelf -d`):** Metadata for the dynamic linker—which shared libraries must be loaded.

B. View 1: The ELF Header (readelf -h)

```
bipin@bipin-VirtualBox:~$ readelf -h procinfo_dyanimc
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 03 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                                   2's complement, little endian
  Version:                             1 (current)
  OS/ABI:                               UNIX - GNU
  ABI Version:                         0
  Type:                                 EXEC (Executable file)
  Machine:                             Advanced Micro Devices X86-64
  Version:                             0x1
  Entry point address:                 0x401900
  Start of program headers:            64 (bytes into file)
  Start of section headers:           981488 (bytes into file)
  Flags:                               0x0
  Size of this header:                 64 (bytes)
  Size of program headers:            56 (bytes)
  Number of program headers:           11
  Size of section headers:            64 (bytes)
  Number of section headers:          28
  Section header string table index: 27
```

Figure 4: Viewing the ELF Header of `procinfo_dynamic`

Key fields:

Magic: The bytes 7f 45 4c 46 (“ELF”) identify this as a valid ELF file.

Class: ELF64 means 64-bit.

Machine: Must match the host CPU. An ARM binary will not execute on x86 64.

Type: EXEC (static) or DYN (position-independent, used for both shared libraries and modern dynamically-linked executables).

C. View 2: The Program Headers (readelf -l)

The Program Headers describe segments—contiguous regions of the file that the OS loader maps into virtual memory.

```
bipin@bipin-VirtualBox:~$ readelf -l procinfo_dynanic

Elf file type is EXEC (Executable file)
Entry point 0x401900
There are 11 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
     FileSiz      MemSiz              Flags   Align
LOAD             0x0000000000000000 0x0000000000400000 0x0000000000400000
                 0x00000000000004f8 0x00000000000004f8  R       0x1000
LOAD             0x0000000000000100 0x0000000000401000 0x0000000000401000
                 0x000000000000a96cd 0x000000000000a96cd  R E     0x1000
LOAD             0x000000000000ab00 0x00000000004ab000 0x00000000004ab000
                 0x00000000000027a08 0x00000000000027a08  R       0x1000
LOAD             0x000000000000d3128 0x00000000004d3128 0x00000000004d3128
                 0x00000000000005918 0x0000000000000b300  RW      0x1000
NOTE             0x00000000000002a8 0x00000000004002a8 0x00000000004002a8
                 0x0000000000000024 0x0000000000000024  R       0x4
NOTE             0x000000000000d29b8 0x00000000004d29b8 0x00000000004d29b8
                 0x0000000000000030 0x0000000000000030  R       0x8
NOTE             0x000000000000d29e8 0x00000000004d29e8 0x00000000004d29e8
                 0x0000000000000020 0x0000000000000020  R       0x4
TLS              0x000000000000d3128 0x00000000004d3128 0x00000000004d3128
                 0x0000000000000038 0x0000000000000050  R       0x8
GNU_PROPERTY     0x000000000000d29b8 0x00000000004d29b8 0x00000000004d29b8
                 0x0000000000000030 0x0000000000000030  R       0x8
GNU_STACK        0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x0000000000000000 0x0000000000000000  RW      0x10
GNU_RELRO        0x000000000000d3128 0x00000000004d3128 0x00000000004d3128
                 0x00000000000003ed8 0x00000000000003ed8  R       0x1

Section to Segment mapping:
Segment Sections...
00  .note.gnu.build-id .rela.plt
01  .init .plt .text .fini
02  .rodata .stapsdt.base rodata.cst32 .eh_frame .gcc_except_table .note.gnu.
03  .tdata .init_array .fini_array .data.rel.ro .got .got.plt .data .bss
04  .note.gnu.build-id
05  .note.gnu.property
```

Figure 5: Viewing the Program Header of `procinfo_dynamic`

Critical entries to look for:

- **INTERP:** Specifies the path to the dynamic linker (e.g., `/lib64/ld-linux-x86-64.so.2`). The kernel reads this and loads the dynamic linker before the program's `main()` runs.
- **LOAD:** Each LOAD segment corresponds to a memory region:
 - R only: typically, `.rodata` (read-only constants).
 - R E: the `.text` segment (executable machine code).
 - RW: the `.data` and `.bss` segments (read-write global variables).

GNU_STACK: Declares the stack's permissions. RW means the stack is readable and writable but not executable—this is the secure default that prevents stack based buffer overflow exploits. If this shows RWE, the stack is dangerously executable.

D. View 3: The Dynamic Section (readelf -d)

This section exists only in dynamically-linked executables. It tells the dynamic linker which shared libraries to load and where to find symbols.

```
bipin@bipin-VirtualBox:~$ readelf -d procinfo_dynamic

Dynamic section at offset 0x2d90 contains 27 entries:
   Tag              Type              Name/Value
0x0000000000000001 (NEEDED)          Shared library: [libc.so.6]
0x000000000000000c (INIT)            0x1000
0x000000000000000d (FINI)            0x1310
0x0000000000000019 (INIT_ARRAY)      0x3d80
0x000000000000001b (INIT_ARRAYSZ)    8 (bytes)
0x000000000000001a (FINI_ARRAY)      0x3d88
0x000000000000001c (FINI_ARRAYSZ)    8 (bytes)
0x0000000000000065 (GNU_HASH)          0x390
0x0000000000000005 (STRTAB)            0x508
0x0000000000000006 (SYMTAB)            0x3b8
0x000000000000000a (STRSZ)             208 (bytes)
0x000000000000000b (SYMENT)            24 (bytes)
0x0000000000000015 (DEBUG)             0x0
0x0000000000000003 (PLTGOT)            0x3f80
0x0000000000000002 (PLTRELSZ)          192 (bytes)
0x0000000000000014 (PLTREL)            RELA
0x0000000000000017 (JMPREL)            0x6f8
0x0000000000000007 (RELA)              0x638
0x0000000000000008 (RELASZ)            192 (bytes)
0x0000000000000009 (RELAENT)           24 (bytes)
0x000000000000001e (FLAGS)             BIND_NOW
0x00000000000000fb (FLAGS_1)           Flags: NOW PIE
0x00000000000000fe (VERNEED)           0x5f8
0x00000000000000ff (VERNEEDNUM)        1
0x00000000000000f0 (VERSYM)            0x5d8
0x00000000000000f9 (RELACOUNT)         3
0x0000000000000000 (NULL)              0x0
bipin@bipin-VirtualBox:~$
```

Figure 6: readelf -d

Critical entries:

NEEDED: Lists every shared library the program requires. The dynamic linker will search for and mmap() each of these into the process's address space before main() begins. If any NEEDED library is missing, the program will refuse to start.

VI. TRACING DEPENDENCIES WITH **L D D**

While `readelf -d` shows the names of required libraries, `ldd` goes further: it actually invokes the dynamic linker to resolve each `NEEDED` entry to a full filesystem path.

```
bipin@bipin-VirtualBox:~$ ldd procinfo_dynamic
linux-vdso.so.1 (0x000078662657e000)
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x0000786626200000)
/lib64/ld-linux-x86-64.so.2 (0x0000786626580000)
```

Figure 7: `ldd` function

What this tells us:

- `libc.so.6` is found at `/lib/x86_64-linux-gnu/libc.so.6`.
- `linux-vdso.so.1` is a virtual shared object injected by the kernel into every process. It provides fast system call entry points without a full context switch.
- `/lib64/ld-linux-x86-64.so.2` is the dynamic linker itself—the program that reads the `INTERP` and `NEEDED` entries and loads everything.

A. Static Binaries and `ldd`

If we run `ldd` on a statically-linked binary, the output confirms that no dynamic dependencies exist:

```
bipin@bipin-VirtualBox:~$ ldd procinfo_static
not a dynamic executable
```

Figure 8: `ldd` function on static

This is because the static binary has no `DYNAMIC` section and no `INTERP` entry in its program headers.

VII. COMMAND REFERENCE SUMMARY

Table II consolidates the key tools, the specific fields to inspect, and what each field tells the operating system.

Command / Term	Look For	What It Tells the OS
readelf -h <file>	Type, Machine	"Is this a valid executable, and does it match my CPU architecture?"
readelf -l <file>	INTERP	"Which dynamic linker must run first to handle shared libraries?"
readelf -l <file>	GNU_STACK	"Is this stack secure (RW) or dangerously executable (RWE)?"
readelf -l <file>	LOAD (Flags)	"Which memory regions are code (R E), read-only data (R), or read-write data (RW)?"
readelf -d <file>	NEEDED	"What external shared libraries must be mapped into memory before <code>main()</code> starts?"
ldd <file>	Full output	"Where exactly on the filesystem are the required <code>.so</code> files located?"
ldd <file>	"not a dynamic executable"	"This binary is statically linked and has no runtime library dependencies."

Figure 9: TABLE II: Command Reference: Inspecting ELF Executables

VIII. CONCLUSION

This lab examined how a practical C program using multiple library functions (`printf`, `getpid`, `time`) links to the C Standard Library, which consists of header files in `/usr/include/` (the construction manual) and precompiled machine code in `/lib/` and `/usr/lib/` (the actual libraries). We contrasted static linking, which produces self-contained but large binaries, with dynamic linking, which produces small executables that share library code in RAM at runtime. We used `readelf` and `ldd` to inspect the three critical views of an ELF executable: the ELF Header (validity and architecture), the Program Headers (memory layout and security), and the Dynamic Section (runtime dependencies).

IX. KNOWLEDGE CHECK QUESTIONS

- 1) What is the fundamental difference between a C header file and a shared library?
 - A header file gives the code instructions to the compiler, while a shared library runs the actual code on the computer.

- 2) What is static and dynamic linking
 - Static linking: Library code is copied into the program at compile time.
Dynamic linking: Library code stays outside and is used when the program runs.

- 3) What is ELF Header? Write down the commands to check headers.
 - The first part of an ELF file (Linux executables and object files) that tells the system what kind of file it is and how to run it.

- 4) What does bookmark means? provide example.
 - A saved shortcut to a folder, so it can be reached without typing the full path.
alias lab="cd ~/Documents/linux_lab" lab

REFERENCES

[1] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Pearson, 2015.

Available: <https://csapp.cs.cmu.edu/3e/perspective.html>

[2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley, 2013.

Available: <https://www.oreilly.com/library/view/advanced-programming-in/9780321638014/>

[3] J. R. Levine, *Linkers and Loaders*. Morgan Kaufmann, 2000.

Available:

https://books.google.com/books/about/Linkers_and_Loaders.html?id=Id9cYsIdjIwC

[4] Tool Interface Standard (TIS), "Executable and Linking Format (ELF) Specification," Version 1.2, 1995.

Available: https://archive.org/details/elf_20210512